hwcost: a calibrated analytical model of compute time and cost

validated on the Pollard rho iteration — results report, 2026-10-16

Study: `research/hwcost_model_20261016/` (protocol `PROTOCOL.md`, frozen at `a9a457ec1` before the run). **Run:** `runs/final`, started 20261016T0459Z UTC; descriptive run report `reports/run-20261016T0459Z/run_report.pdf`. **Code:** `src/cryptanalysis/hwcost/`, CLI `src/bin/hwcost.rs`.

1. What the tool does

A hardware profile and a workload go in; a time and cost breakdown comes out.

- **Compute**: iterations × Σ (operations per iteration × cycles per operation) / clock. Cycles per operation come from the profile: measured on the hot loop by `hwcost measure`, or entered from instruction tables, llvm-mca or uiCA with the source named. An operation on a serial chain is priced at its latency (`op@latency`), the rest at reciprocal throughput.
- **Memory**: bytes moved per iteration / bandwidth of the first level that holds the per-core working set; a phase takes the larger of compute and memory time (the roofline, applied to a CPU).
- **Parallelism**: Amdahl with the **measured** scaling efficiency, interpolated between measured core counts and flagged when extrapolated.
- **Transfers**: messages × latency + bytes / bandwidth per data movement; transfers that overlap compute take a max with it, others add.
- **Cost**: wall hours × machines × $/h, plus egress and storage.

Every constant carries its source; every prediction lists the unmeasured constants it rests on. An operation the workload names but the profile does not price is an error, never a zero.

2. Calibration and validation on the rho iteration

The rho kernel (`hwcost::kernel`) advances `B` affine walks of an r-adding walk together with one Montgomery batch inversion. Its operation counts per iteration were counted, not asserted: 3 multiplications with independent work around them, 2 on the batch inversion's prefix-product chain, 1 squaring, 6 additions/subtractions and 1/B inversion (counted step on the timed walks, every field and batch size). Four predictions were compared with the measured median ns per iteration (protocol, "Models"): **A** a priori with no fitted parameter; **C** = A plus one per-iteration residual per field fitted on B ≤ 1024 and tested on larger B; **S** for `w4` (P-256 field) with every operation cost fitted against limb count on 1–3 limbs and extrapolated.

claim	statement	value	threshold	verdict
C0	clock: add-chain and imul-chain estimates agree, |relative difference|	0.0281	≤ 0.05	**PASS**
V1.fp64	fp64: calibrated model, test batches, median |error|	0.0271	≤ 0.3	**PASS**
V1.w1	w1: calibrated model, test batches, median |error|	0.0549	≤ 0.3	**PASS**
V1.w2	w2: calibrated model, test batches, median |error|	0.0372	≤ 0.3	**PASS**
V1.w3	w3: calibrated model, test batches, median |error|	0.0175	≤ 0.3	**PASS**
V1.w4	w4: calibrated model, test batches, median |error|	0.0283	≤ 0.3	**PASS**
V2.fp64	fp64: a priori model (no fitted parameter), all batches, median |error|	0.542	≤ 0.3	**FAIL**
V2.w1	w1: a priori model (no fitted parameter), all batches, median |error|	0.03	≤ 0.3	**PASS**
V2.w2	w2: a priori model (no fitted parameter), all batches, median |error|	0.1811	≤ 0.3	**PASS**
V2.w3	w3: a priori model (no fitted parameter), all batches, median |error|	0.1237	≤ 0.3	**PASS**
V2.w4	w4: a priori model (no fitted parameter), all batches, median |error|	0.1356	≤ 0.3	**PASS**
V3.w4	w4: size-extrapolated model (fitted on smaller limb counts), all batches, median |error|	0.0744	≤ 0.3	**PASS**

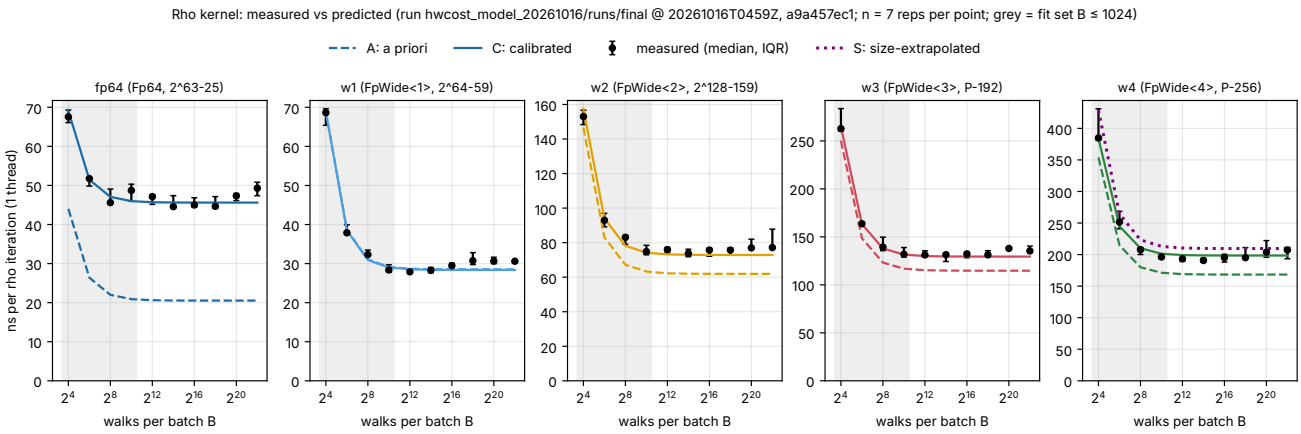


Figure 1: Measured (median, IQR, n = 7) and predicted ns per rho iteration against batch size; grey is the fit set. Run `runs/final`, 20261016T0459Z.

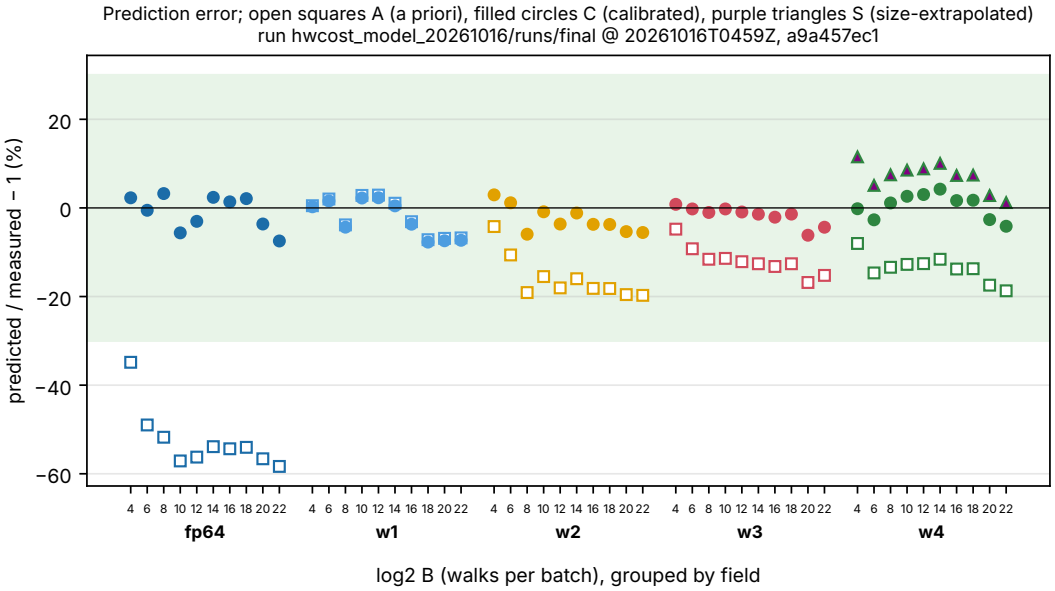


Figure 2: Prediction error per field and batch size; band ±30%.

2.1. What the results say

- **Calibrated (C): every field passes**, with median absolute error between 1.7% and 5.5% on the test batches, 4× to 4096× larger than the largest batch the residual was fitted on. The worst single test row is 7.7%. One fitted constant per field is enough for this kernel on this host, in line with the 10–30% expectation for a calibrated compute-bound model, at the low end.
- **A priori (A): 4 of 5 fields pass, `fp64` fails** (median |error| 54.2%). On the `FpWide` fields A's error lies between −19.7% and 2.9% (it under-predicts `w2`–`w4` at every batch size); on `fp64` it lies between −58.3% and −34.8%: the microbenchmarked operation costs explain only about half of the measured time there. The fitted residuals (measured − A, ns per iteration): `fp64` 25.1, `w1` −0.2, `w2` 10.9, `w3` 14.7, `w4` 30.2. The `fp64` backend's operations are cheaper than `w1`'s in isolation (mul 1.28 vs 2.21 ns throughput), yet its kernel is about 1.6× slower (≈ 45 vs ≈ 28 ns per iteration). Its operations are inlined (no out-of-line `Fp64::mul/sub` symbol), so call overhead is excluded. **Unverified lead:** `Fp64::sub` and `Fp64::redc` take data-dependent `if` branches that a microbenchmark's single chain may predict but random walk data may not; no `perf` counters were available to check. This is exactly the gap the calibration step exists to close.
- **Size extrapolation (S)**: fitting every operation cost against limb count on 1–3 limbs and the residual linearly, then extrapolating to 4 limbs, predicts the P-256-field kernel with median |error| 7.4% (over-predicting by 1–12%). The fitted costs at n = 4 come out above the measured ones (fig. 4 of the run report: e.g. mul throughput extrapolated 26.4 ns vs measured 23.6 ns).
- **Memory roofline**: the model assigns every arm to compute (the kernel draws at most ≈ 3.5 GB/s against ≥ 8.8 GB/s single-thread DRAM read bandwidth), and the measurement agrees to first order: once the inversion is amortized, growing the working set from L2 to beyond the last-level cache moves the per-iteration time by 2–10% depending on the field (B = 2^16 to 2^22). That rise is a cost the `max()` roofline does not represent, and C's error drifts negative there (down to −7.7%).

- **Clock**: the reg-reg `add` and `imul` chains agree within 2.8% at 2.878 GHz (the nominal clock is 2.1 GHz). A chain of `add reg, imm` reads about 12 GHz on this CPU: dependent immediate adds are folded at rename, so such a chain is not a clock measurement.
- **Scaling**: efficiency at 4 threads `fp64` 0.889, `w1` 0.903, `w2` 0.942, `w3` 0.939, `w4` 0.826 (L0 host; the 4 vCPUs may share physical cores). The profile carries the median over fields.

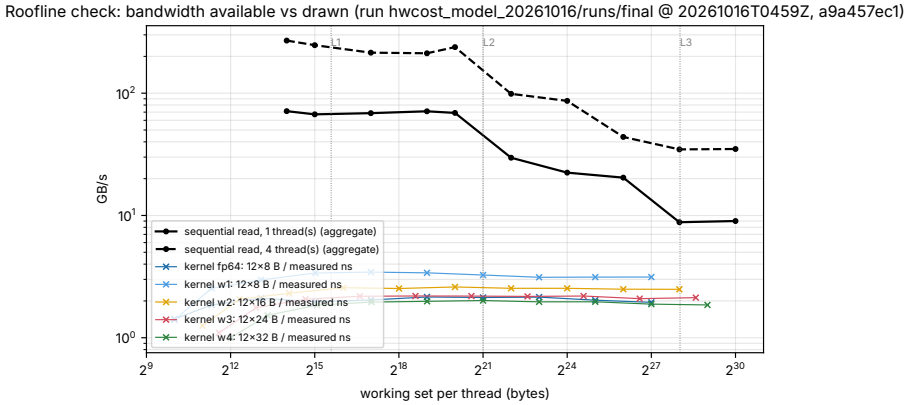


Figure 3: Bandwidth available (sequential read) vs bandwidth the kernel drew.

3. Application: parallel rho on prime-order curves (descriptive, no claim)

`hwcost rho` turns the kernel's counted operations into a van Oorschot–Wiener workload. It uses $\sqrt{(\pi n/2)}$ expected iterations ($\sqrt{(\pi n/4)}$ with the negation map, fruitless cycles unpriced) plus the distinguished-point tail. Distinguished-point bits are chosen so the tail stays ≤ 1%. Start points are about 1.5 $\log_2 n$ group operations per walk. Distinguished points are uploaded over each machine's link and stored for the run. The workload is predicted on the fitted profile at B = 1024 walks per core, 4 cores per machine, with the field the smallest measured one holding the group. The price is illustrative (AWS us-east-1 on-demand c7i.xlarge, $0.1785/h); the network and disk constants are assumed (the output lists them).

$\log_2 n$	field	iterations ($\log_2$)	ns/iter	1 machine	1,000 machines	cost, 1 machine ($)	same, negation map ($)
48	fp64	24.33	46	662.623 ms	25.177 ms	< 0.01	< 0.01
64	w1	32.33	29	43.498 s	678.428 ms	< 0.01	< 0.01
80	w2	40.33	74.2	7.91 h	28.485 s	1.41	1
96	w2	48.33	74.2	84.38 days	2.02 h	361.49	256.27
112	w2	56.33	74.2	59.14 years	21.60 days	92541.76	65604.73
128	w2	64.33	74.2	2^13.9 years	15.14 years	2^24.5	2^24
160	w3	80.33	131.6	2^30.7 years	2^20.7 years	2^41.3	2^40.8
192	w3	96.33	131.6	2^46.7 years	2^36.7 years	2^57.3	2^56.8
224	w4	112.33	201.4	2^63.3 years	2^53.4 years	2^73.9	2^73.4
256	w4	128.33	201.4	2^79.3 years	2^69.4 years	2^89.9	2^89.4

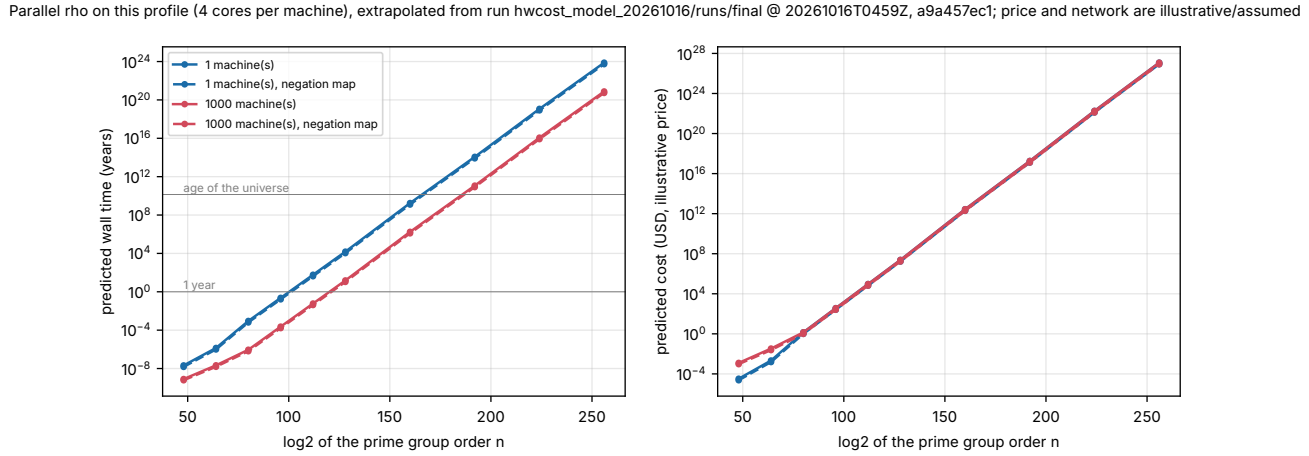


Figure 4: Predicted wall time and cost of parallel rho against group size on this profile (4 cores per machine). Extrapolated from run `runs/final`; price and network illustrative/assumed.

Reading the table: the time per iteration grows with the limb count of the field, so cost tracks √n × (cost of one field's arithmetic). At n = 2^48 on 1,000 machines the start-point setup dominates (4 million walks at about 72 group operations each, against 2^24.6 walk iterations), so 1,000 machines are only 26× faster than one. At 2^112 (the size of the solved secp112r1 instance) the model gives about 3 weeks on 1,000 four-core machines; at 2^256 (P-256) it gives about 2^69 years on the same fleet. Both are extrapolations of a 3-minute calibration of one 4-vCPU host and one kernel; they inherit every limit below.

4. Scope and limits

- **One host, isolation L0** (shared 4-vCPU KVM guest, no pinning, turbo uncontrolled), **one kernel**, **one build** (x86-64 baseline: no BMI2/ADX/AVX2 code generation even though the CPU has them). An ISA-specific build is a different profile and needs its own `hwcost measure` run.
- The rho kernel does not restart walks at distinguished points and solves no logarithm; the application's DP tail, start points, network and storage terms are modelled, not measured.
- GPU profiles are supported by the schema (SMs as cores, instruction classes as operations) but none was measured here.
- Not an index-calculus result, so no gap-to-rho row; no two ECDLP methods are ranked, so the AGENTS.md §5a unpriced-work band does not apply.
- Correction during the study: the application first capped distinguished-point bits at 64, which for groups above 2^192 left up to 2^64 points stored and made storage dominate the cost. The cap was removed before the reported `rho*.csv` were produced (commit after `a9a457ec1` kernel and measurement code unchanged).

5. Next steps

1. Run `hwcost measure` on a `-C target-cpu=native` build (MULX/ADX, AVX-512 IFMA `Fp52`) and compare profiles: the ISA effect on these field operations.
2. Check the `fp64` lead with hardware counters (branch misses) on a host with `perf`; if confirmed, a branch-free `Fp64::sub`/`redc` is an engineering change to measure on the rho kernel.
3. Add a GPU profile from the measured pipe ceilings in `benchmarks/hardware-limits/` and the ECC2K-130 per-pipe instruction counts (`ecc2k130/ROOFLINE.md`), and validate it against the recorded 15.1 B it/s.
4. Price an index-calculus workload phase by phase (relation search, linear algebra, descent) from ecbench's counted units, so IC and rho are compared in time and dollars as well as in GAE.